The Relay, v2: resubmission

In short: I added embeddings to retrieval, moved citations into metadata, made answers stream so the first sentence arrives sooner, and turned the pipeline into a service with a Redis cache. I also wrote down the constraints I had only implied in the first submission.

The first submission is unchanged: tag `submission-2026-11-12`, folder `submission/`. This document covers what changed after the interview, why, and what each change measured, including what didn't improve. Every number here comes from a file a script produced:

- `results_v2.csv` and `tables_v2.md` (`scripts/build_results_v2.py`);
- `data/retrieval/v2.md` (`scripts/eval_retrieval_v2.py`).

1. What the review found, and what changed

Feedback	Change	Measured
No embeddings; BM25 alone	Hybrid retrieval: BM25 + bge-small-en-v1.5 embeddings, merged by reciprocal rank fusion; the router writes a hypothetical answer (HyDE) to steer the embedding search	On plain-wording questions the answer is retrieved **78%** of the time (BM25: **50%**)
No hierarchical chunking	Small-to-big: search 600-char children, give the drafter their 2400-char parent section	Part of the row above; see section 3
Constraints not stated	Written out with numbers (section 2)	—
Why parse citations instead of carrying metadata?	The drafter returns `{text, sources, quote}` per sentence; the system writes the citations	All citation-parsing failures gone; see section 4
First token = total time	Each sentence is released as soon as its check passes	First verified sentence p50 **1.46 s**
A script, not a service	FastAPI + Redis (cache, single-flight, fleet-wide rate limits, job queue), worker, Prometheus, Docker	20 requests over 4 distinct questions cost **4** drafts

2. The constraints, inferred from the brief

Goal	Constraint
Never wrong	No sentence is sent unless it is backed by text in a cited source (its quote is found there, its numbers too). If the papers don't answer, say so; never guess.
Fast	First sentence ≤ ~3 s at p50 when budget allows; no one waits more than 15 s for budget; every request finishes within 30 s.
Nobody turned away	Every request gets a full answer, a shorter one, or cited passages. Never an error or a hang.
Cheap	$0 on the free tier; ≤ ~$2 per 1,000 answers at paid prices; ≤ ~3,500 tokens per answer.
Inside provider limits	Never above the measured per-minute and per-day limits of each model, **counted across all servers**; no retry storms.
Honest under load	A weaker answer says it is weaker and why; no wait time is promised that can't be backed up.

3. Retrieval: embeddings, hybrid, small-to-big

The test set was biased. My 14 original questions used the papers' own words, so BM25 retrieved the answer for 93% of them. I added 32 questions phrased the way an engineer would ask (`data/retrieval/dev_questions.jsonl`). On those, the submitted BM25 retrieved only 50%.

Measured with no model calls, raw question as the query, 46 answerable questions:

Retrieval	Pieces read	All 46	Original 14 (hit / fact recall)	Plain-wording 32	Context
BM25, submitted	3	63%	93% / 84%	50%	5.6k chars
Embeddings only	3	67%	71% / 50%	66%	5.3k
Small-to-big hybrid	3	76%	71% / 66%	78%	5.9k
Hybrid + cross-encoder rerank	3	80%	86% / 77%	78%	1.8 s/query on CPU
Chosen: BM25 top 3 kept + small-to-big hybrid	**5**	**83%**	**93% / 88%**	**78%**	9.3k

With only three slots, fusion pushed out passages BM25 had ranked 2nd or 3rd, and those held exact-term facts (Q11, Q16). The chosen setting keeps BM25's top three and lets fusion fill up to five. It loses nothing on the original questions and gains 28 points on plain wording, at the cost of about 900 more prompt tokens per draft.

The reranker is off by default because it costs ~1.7 s per query on a CPU; on a GPU host it should be on. Q14 ("where does H-Net cut the byte stream?") is still not retrieved by any variant: its gold passage was extracted as equation fragments (section 7).

4. Citations carried as metadata

The drafter now answers in JSON Lines and never writes citation markup:

```
{"text": "The base model uses a dropout rate of 0.1.", "sources": [1], "quote": "For the base model, we use a rate of P_drop = 0.1"}
```

Each claim is checked with string operations as soon as its line arrives:

- the sources it names were given;
- its quote occurs in a cited source;
- its numbers appear there.

What this removes: every real failure of the v1 citation check was a *parsing* failure (`【S3】`, zero-width spaces in `[S1]`, formulas split into uncited lines, `{0,1}` read as one number). With metadata there's nothing to parse. A bad claim is also withheld on its own instead of failing the whole answer.

What it doesn't remove: checking. A model can attach the wrong source, or a real quote that doesn't support its sentence. The quote turns the first case into a free check; the second still needs the LLM verifier, which runs when the answer isn't streamed.

5. Speed: streaming verified sentences

Each claim is released the moment it passes its check: over Server-Sent Events from the API, or straight into text-to-speech in a voice product. Real Groq, gpt-oss-20b, p50 excluding budget wait:

Pipeline	First sentence p50 / p95	Complete p50 / p95
v2 claims (streams)	**1,459 / 3,319 ms**	1,648 / 3,320 ms
v2 retrieval + prose (buffered)	1,795 / 2,926 ms	1,795 / 2,926 ms

The trade-off: the LLM verifier needs the whole answer, so a streamed answer relies on the per-claim quote check, and the reply says so.

6. The service

Part	What it does
API (FastAPI)	`/v1/ask`, `/v1/ask/stream` (SSE), `/v1/jobs`, `/healthz`, `/readyz`, `/metrics`; API keys and a per-client rate limit
Redis cache + single-flight	The same question (normalised) is answered once: later requests get the cached answer; requests that arrive while it's being drafted wait for it
Fleet-wide rate limit	One token bucket per model in Redis (atomic Lua script), from the measured limits. Every server must take its share before calling Groq, so two replicas can't each spend the whole budget
Job queue	Redis Streams with consumer groups; a crashed worker's jobs are redelivered
Metrics	Prometheus: first-token and total latency, cache hits, model calls by status, tokens, withheld claims
Deployment	Docker Compose: Redis, 2 API replicas behind nginx, worker, stand-in model, optional Prometheus

Kafka isn't used: the service needs a cache, counters, locks and a small queue, and Redis does all four. Kafka fits a high-volume event log replayed by many consumers; the events stream is shaped so it can be mirrored to Kafka later.

Measured on the running stack (stand-in model, so speed and behaviour, not answer quality):

- 20 simultaneous requests over 4 distinct questions → 4 drafts, 16 shared.
- Repeats served from the cache in under 1 ms.
- The worker survived a Redis restart.

Two bugs found only by running it:

- The embedding model wasn't actually in the image, so each replica downloaded it on its first request: 16–18 s. Fixed; the first request is now 1.1–1.8 s.
- An idle worker crashed on a Redis read timeout. Fixed with timeouts and back-off.

7. Answer quality: honest results

Real Groq, the same 18 graded questions as the first submission, auto-grades, verifier off. One question = 5.6 points.

Pipeline	Drafter	Correct + cited	Wrong	Correctly refused	Tokens/answer
First submission (auto-grades)	gpt-oss-20b	44.4%	33.3%	22.2%	–
v2 retrieval + prose	gpt-oss-20b	**50.0%**	27.8%	22.2%	3,797
v2 claims (default)	gpt-oss-20b	44.4%	33.3%	22.2%	**2,935**
First submission (Table 3)	gpt-oss-120b	72.2% (auto 66.7%)	5.6%	22.2%	1,948
v2 claims, before the fixes below	gpt-oss-120b	16.7% → 33.3%	—	22.2%	~3,000

What this shows: on these 18 questions, answer accuracy is **on par, not better**; the differences are one question either way. The 18 are mostly in the papers' own words, where BM25 was already strong, so the retrieval gain shows up on the plain-wording questions instead, which don't have graded answers yet. Claims mode streams and uses 23% fewer tokens than prose, for one question less.

How the first 120b runs went wrong, and how I found it:

- Answers were grounded but incomplete. Raw drafts showed true claims being withheld because the PDF extraction glues words together ("whereF represents"), so correct quotes failed a word-by-word match.
- gpt-oss-20b also wrote several JSON objects on one line, and quoted tables skipping cells.

All three are fixed and covered by tests. **gpt-oss-120b wasn't re-run after the fixes** (daily budget), so its two numbers measure the bugs, not the final design. `[draft] format = "text"` switches to prose in one line.

8. What's next, in order

1. A layout-aware PDF parser. Equations and tables come out as fragments; that's the root of Q14 and of the quote-matching problems.
2. Graded answers for the plain-wording questions, written by someone who hasn't seen the passages.

3. The reranker on GPU.
4. A router in JSON mode, and paper selection without a model call.

9. How to run and reproduce

```
docker compose up --build                      API on http://localhost:8080 (no key: stand-in model)
python -m unittest discover -s tests           197 tests, no key, no network, no Redis
python scripts/eval_retrieval_v2.py            retrieval tables (no model calls)
python scripts/eval_v2_quality.py --real       answer quality on real Groq (spends budget)
python scripts/build_results_v2.py             results_v2.csv and tables_v2.md from the saved runs
python ask.py "your question" --real --stream  one question from the terminal
```

Design and full critique of every step: `docs/V2_DESIGN.md`. Code: `code_v2.zip`, or https://github.com/Dishan18/Vaani_Relay (main). The first submission: tag `submission-2026-11-12`.

10. How I worked with the assistant

The v2 work was done with Claude Code after the interview. I set the direction:

- address each point of the review;
- carry citations as metadata (the reviewer's question);
- make it production-shaped, with Redis and whatever else was needed;
- rethink every step, not just retrieval.

The assistant measured before changing defaults, and found the quality regressions on real Groq runs rather than the stand-in. They were traced through the raw drafts to PDF extraction artifacts, a number check that split "0.1" into two numbers, and parsing assumptions, and each one is now a test. What it couldn't do was re-run gpt-oss-120b after the fixes, so that result is open. Every run is reported, including the bad ones: the results are "on par, not better", and that is what this document says.